

条款 45：运用成员函数模板接受所有兼容类型

Use member function templates to accept "all compatible types."

所谓智能指针 (*Smart pointers*) 是“行为像指针”的对象，并提供指针没有的机能。例如条款 13 曾经提及 `std::auto_ptr` 和 `tr1::shared_ptr` 如何能够被用来在正确时机自动删除 heap-based 资源。STL 容器的迭代器几乎总是智能指针；无疑你不会奢望使用 “++” 将一个内置指针从 linked list 的某个节点移到另一个节点，但这在 `list::iterators` 身上办得到。

真实指针做得很好的一件事是，支持隐式转换 (*implicit conversions*)。Derived class 指针可以隐式转换为 base class 指针，“指向 non-const 对象”的指针可以转换为“指向 const 对象”……等等。下面是可能发生于三层继承体系的一些转换：

```
class Top { ... };
class Middle: public Top { ... };
class Bottom: public Middle { ... };
Top* pt1 = new Middle;           //将 Middle* 转换为 Top*
Top* pt2 = new Bottom;           //将 Bottom* 转换为 Top*
const Top* pct2 = pt1;           //将 Top* 转换为 const Top*
```

但如果想在用户自定的智能指针中模拟上述转换，稍稍有点麻烦。我们希望通过下代码通过编译：

```
template<typename T>
class SmartPtr {
public:                           //智能指针通常
    explicit SmartPtr(T* realPtr); //以内置（原始）指针完成初始化
    ...
};

SmartPtr<Top> pt1 =                //将 SmartPtr<Middle> 转换为
    SmartPtr<Middle>(new Middle); // SmartPtr<Top>
SmartPtr<Top> pt2 =                //将 SmartPtr<Bottom> 转换为
    SmartPtr<Bottom>(new Bottom); // SmartPtr<Top>
SmartPtr<const Top> pct2 = pt1;    //将 SmartPtr<Top> 转换为
                                   // SmartPtr<const Top>
```

但是，同一个 `template` 的不同具现体 (*instantiations*) 之间并不存在什么与生俱来的固有关系（译注：这里意指如果以带有 base-derived 关系的 B、D 两类型分别具现化某个 `template`，产生出来的两个具现体并不带有 base-derived 关系），所以编译器视 `SmartPtr<Middle>` 和 `SmartPtr<Top>` 为完全不同的 classes，它们之间的关系并不比……唔……并不比 `vector<float>` 和 `Widget` 更密切，呵呵。为了获得我们希望获得的 `SmartPtr` classes 之间的转换能力，我们必须将它们明确地编写出来。

Templates 和 泛型编程 (Generic Programming)

在上述智能指针实例中, 每一个语句创建了一个新式智能指针对象, 所以现在我们应关注如何编写智能指针的构造函数, 使其行为能够满足我们的转型需要。一个很具关键的观察结果是: 我们永远无法写出我们需要的所有构造函数。在上述继承体系中, 我们根据一个 `SmartPtr<Middle>` 或一个 `SmartPtr<Bottom>` 构造出一个 `SmartPtr<Top>`, 但如果这个继承体系未来有所扩充, `SmartPtr<Top>` 对象又必须能够根据其他智能指针构造自己。假设日后添加了:

```
class BelowBottom: public Bottom { ... };
```

我们因此必须令 `SmartPtr<BelowBottom>` 对象得以生成 `SmartPtr<Top>` 对象, 但我们当然不希望一再修改 `SmartPtr` **template** 以满足此类需求。

就原理而言, 此例中我们需要的构造函数数量没有止尽, 因为一个 **template** 可被无限量具现化, 以致生成无限量函数。因此, 似乎我们需要的不是为 `SmartPtr` 写一个构造函数, 而是为它写一个构造模板。这样的模板 (**templates**) 是所谓 *member function templates* (常简称为 *member templates*), 其作用是为 **class** 生成函数:

```
template<typename T>
class SmartPtr {
public:
    template<typename U>                //member template,
    SmartPtr(const SmartPtr<U>& other);  //为了生成 copy 构造函数
    ...
};
```

以上代码的意思是, 对任何类型 **T** 和任何类型 **U**, 这里可以根据 `SmartPtr<U>` 生成一个 `SmartPtr<T>` —— 因为 `SmartPtr<T>` 有个构造函数接受一个 `SmartPtr<U>` 参数。这一类构造函数根据对象 **u** 创建对象 **t** (例如根据 `SmartPtr<U>` 创建一个 `SmartPtr<T>`), 而 **u** 和 **v** 的类型是同一个 **template** 的不同具现体, 有时我们称之为泛化 (*generalized*) *copy* 构造函数。

上面的泛化 *copy* 构造函数并未被声明为 `explicit`。那是蓄意的, 因为原始指针类型之间的转换 (例如从 **derived class** 指针转为 **base class** 指针) 是隐式转换, 无需明白写出转型动作 (`cast`), 所以让智能指针仿效这种行径也属合理。在模板化构造函数 (**templated constructor**) 中略去 `explicit` 就是为了这个目的。

完成声明之后, 这个为 `SmartPtr` 而写的“泛化 *copy* 构造函数”提供的东西比我们需要的更多。是的, 我们希望根据一个 `SmartPtr<Bottom>` 创建一个 `SmartPtr<Top>`, 却不希望根据一个 `SmartPtr<Top>` 创建一个 `SmartPtr<Bottom>`, 因为那对 **public** 继承而言 (见条款 32) 是矛盾的。我们也不希望根据一个

SmartPtr<double> 创建一个 SmartPtr<int>, 因为现实中并没有“将 int* 转换为 double*” 的对应隐式转换行为。是的, 我们必须从某方面对这一 member template 所创建的成员函数群进行拣选或筛除。

假设 SmartPtr 遵循 auto_ptr 和 tr1::shared_ptr 所提供的榜样, 也提供一个 get 成员函数, 返回智能指针对象 (见条款 15) 所持有的那个原始指针的副本, 那么我们可以在“构造模板”实现代码中约束转换行为, 使它符合我们的期望:

```
template<typename T>
class SmartPtr {
public:
    template<typename U>
    SmartPtr(const SmartPtr<U>& other)    //以 other 的 heldPtr
        : heldPtr(other.get()) { ... }    // 初始化 this 的 heldPtr
    T* get() const { return heldPtr; }
    ...
private:
    T* heldPtr;                        //这个 SmartPtr 持有的内置 (原始) 指针
};
```

我使用成员初值列 (member initialization list) 来初始化 SmartPtr<T> 之内类型为 T* 的成员变量, 并以类型为 U* 的指针 (由 SmartPtr<U> 持有) 作为初值。这个行为只有当“存在某个隐式转换可将一个 U* 指针转为一个 T* 指针”时才能通过编译, 而那正是我们想要的。最终效益是 SmartPtr<T> 现在有了一个泛化 copy 构造函数, 这个构造函数只在其所获得的实参隶属适当 (兼容) 类型时才通过编译。

member function templates (成员函数模板) 的效用不限于构造函数, 它们常扮演的另一个角色是支持赋值操作。例如 TR1 的 shared_ptr (见条款 13) 支持所有“来自兼容之内置指针、tr1::shared_ptrs、auto_ptrs 和 tr1::weak_ptrs (见条款 54)”的构造行为, 以及所有来自上述各物 (tr1::weak_ptrs 除外) 的赋值操作。下面是 TR1 规范中关于 tr1::shared_ptr 的一份摘录, 其中强烈倾向声明 template 参数时采用关键字 class 而不采用 typename (条款 42 曾说过, 两者的意义在此语境下完全相同)。

```
template<class T>
class shared_ptr {
public:
    template<class Y>                                //构造, 来自任何兼容的
    explicit shared_ptr(Y* p);                        //内置指针、
    template<class Y>
    shared_ptr(shared_ptr<Y> const& r);                //或 shared_ptr、
    template<class Y>
    explicit shared_ptr(weak_ptr<Y> const& r);        //或 weak_ptr、
    template<class Y>
    explicit shared_ptr(auto_ptr<Y>& r);              //或 auto_ptr.
```

```

template<class Y>                                //赋值, 来自任何兼容的
    shared_ptr& operator=(shared_ptr<Y> const& r);    //shared_ptr、
template<class Y>                                //或auto_ptr.
    shared_ptr& operator=(auto_ptr<Y>& r);
...
};

```

上述所有构造函数都是 `explicit`, 惟有“泛化 *copy* 构造函数”除外。那意味从某个 `shared_ptr` 类型隐式转换至另一个 `shared_ptr` 类型是被允许的, 但从某个内置指针或从其他智能指针类型进行隐式转换则不被认可 (如果是显式转换如 `cast` 强制转型动作倒是可以)。另一个趣味点是传递给 `tr1::shared_ptr` 构造函数和 *assignment* 操作符的 `auto_ptr`s 并未被声明为 `const`, 与之形成对比的则是 `tr1::shared_ptr`s 和 `tr1::weak_ptr`s 都以 `const` 传递。这是因为条款 13 说过, 当你复制一个 `auto_ptr`s, 它们其实被改动了。

member function templates (成员函数模板) 是个奇妙的东西, 但它们并不改变语言基本规则。条款 5 说过, 编译器可能为我们产生四个成员函数, 其中两个是 *copy* 构造函数和 *copy assignment* 操作符。现在, `tr1::shared_ptr` 声明了一个泛化 *copy* 构造函数, 而显然一旦类型 `T` 和 `Y` 相同, 泛化 *copy* 构造函数会被具现化为“正常的” *copy* 构造函数。那么究竟编译器会暗自为 `tr1::shared_ptr` 生成一个 *copy* 构造函数呢? 或当某个 `tr1::shared_ptr` 对象根据另一个同型的 `tr1::shared_ptr` 对象展开构造行为时, 编译器会将“泛化 *copy* 构造函数模板”具现化呢?

一如我所说, *member templates* 并不改变语言规则, 而语言规则说, 如果程序需要一个 *copy* 构造函数, 你却没有声明它, 编译器会为你暗自生成一个。在 `class` 内声明泛化 *copy* 构造函数 (是个 *member template*) 并不会阻止编译器生成它们自己的 *copy* 构造函数 (一个 *non-template*), 所以如果你想要控制 *copy* 构造的方方面面, 你必须同时声明泛化 *copy* 构造函数和“正常的” *copy* 构造函数。相同规则也适用于赋值 (*assignment*) 操作。下面是 `tr1::shared_ptr` 的一份定义摘要, 例证上述所言:

```

template<class T>
class shared_ptr {
public:
    shared_ptr(shared_ptr const& r);                //copy 构造函数.
    template<class Y>
        shared_ptr(shared_ptr<Y> const& r);        //泛化 copy 构造函数.
    shared_ptr& operator=(shared_ptr const& r);    //copy assignment.
    template<class Y>
        shared_ptr& operator=(shared_ptr<Y> const& r);    //泛化 copy assignment.
    ...
};

```

请记住

- 请使用 `member function templates` (成员函数模板) 生成“可接受所有兼容类型”的函数。
- 如果你声明 `member templates` 用于“泛化 `copy` 构造”或“泛化 `assignment` 操作”，你还是需要声明正常的 `copy` 构造函数和 `copy assignment` 操作符。

条款 46：需要类型转换时请为模板定义非成员函数

Define non-member functions inside templates when type conversions are desired.

条款 24 讨论过为什么惟有 `non-member` 函数才有能力“在所有实参身上实施隐式类型转换”，该条款并以 `Rational class` 的 `operator*` 函数为例。我强烈建议你继续看下去之前先让自己熟稔那个例子，因为本条款首先以一个看似无害的改动扩充条款 24 的讨论；本条款将 `Rational` 和 `operator*` 模板化了：

```
template<typename T>
class Rational {
public:
    Rational(const T& numerator = 0,           //条款 20 告诉你为什么参数以
            const T& denominator = 1);        // passed by reference 方式传递。
    const T numerator() const;                 //条款 28 告诉你为什么返回值
    const T denominator() const;              // 以 passed by value 方式传递。
    ...                                         //条款 3 告诉你为什么它们是 const.
};

template<typename T>
const Rational<T> operator* (const Rational<T>& lhs,
                             const Rational<T>& rhs)
{ ... }
```

就像条款 24 一样，我们希望支持混合式 (`mixed-mode`) 算术运算，所以我们希望以下代码顺利通过编译。我们也预期它会，因为它正是条款 24 所列的同一份代码，唯一不同的是 `Rational` 和 `operator*` 如今都成了 `templates`：

```
Rational<int> oneHalf(1, 2);                  //这个例子来自条款 24，
                                              //唯一不同是 Rational 改为 template。
Rational<int> result = oneHalf * 2;           //错误！无法通过编译。
```

上述失败给我们的启示是，模板化的 `Rational` 内的某些东西似乎和其 `non-template` 版本不同。事实的确如此。在条款 24 内，编译器知道我们尝试调用什么函数（就是接受两个 `Rationals` 参数的那个 `operator*` 啦），但这里编译器不知道我们想要调用哪个函数。取而代之的是，它们试图想出什么函数被名为 `operator*`